PPO Hyperparameter Analysis on LunarLander-v3 (Continuous)

Zhang Han

March 3, 2026

1 Experimental Setup and Baseline

To analyze the effect of different hyperparameters on the Proximal Policy Optimization (PPO) agent, we employed a control variable approach. The baseline configuration was set as follows:

- `Learning Rate` = 0.0003
- `Gamma` = 0.99
- `GAE Lambda` = 0.95
- `Clip Range` = 0.2
- `Network Architecture` = [64, 64]

Baseline Observation (`arch64_64`): As shown in Figure 1, the baseline model demonstrates a steady learning curve. It successfully breaks the 0-reward threshold around episode 400 and gradually converges towards a reward of 200. However, the evaluation curve exhibits some variance even in later episodes, occasionally dipping below 100, suggesting room for optimization in stability and learning efficiency.

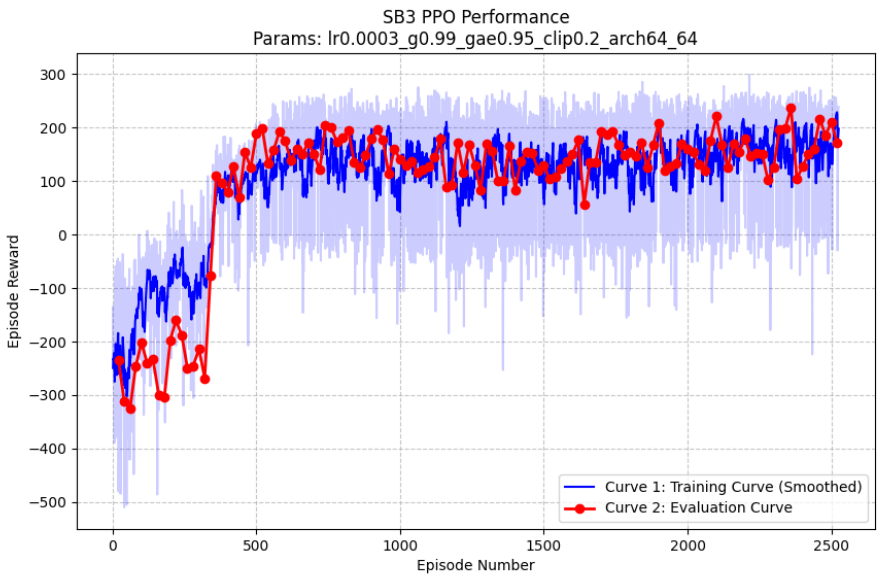

Figure 1: Baseline performance curve (Arch [64, 64], LR 0.0003, Gamma 0.99).

2 Effect of Learning Rate (LR)

Experiment: Increased LR from `0.0003` to `0.001`.

Observation (`lr0.001`): A higher learning rate significantly accelerates the initial learning phase (see Figure 2). The agent reaches high rewards (around 200) by episode 250, much faster than the baseline. However, this comes at the cost of **extreme instability**. The evaluation curve oscillates violently, frequently crashing back down to negative rewards.

Conclusion: An LR of 0.001 is too aggressive for this continuous environment, causing the policy updates to overshoot the optimal parameters and leading to "catastrophic forgetting."

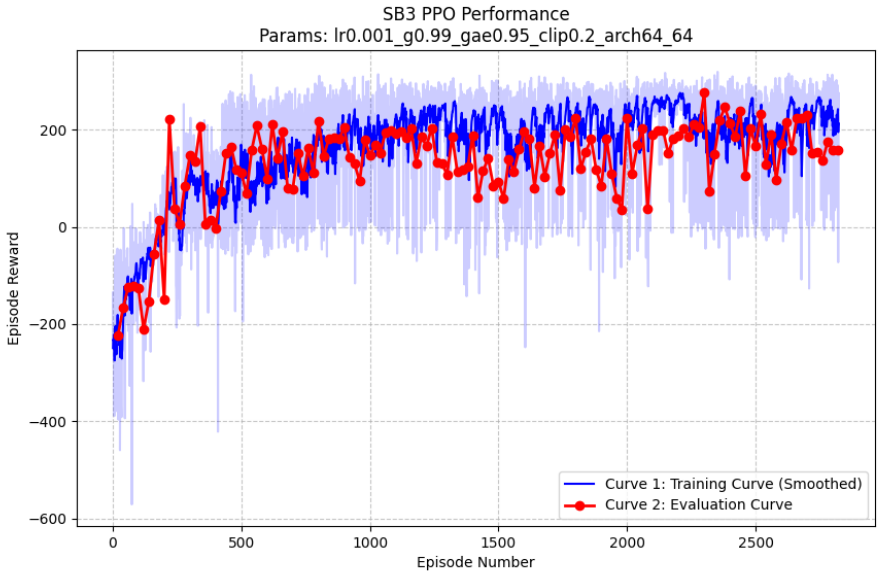

Figure 2: Performance with an aggressive learning rate of 0.001.

3 Effect of Discount Factor (Gamma)

Experiment: Increased Gamma from `0.99` to `0.999`.

Observation (`gamma0.999`): A higher gamma forces the agent to care more about long-term future rewards rather than immediate gains. In Figure 3, we can observe that this makes the learning process **significantly more difficult and slower**. While the baseline crosses the 0-reward threshold around episode 400, the `0.999` agent struggles extensively in the early stages, making the initial climb extremely arduous. Although it might achieve stability if trained for a much longer period, the immediate learning efficiency is severely impacted.

Conclusion: While a gamma of 0.999 might theoretically offer better long-term behavior, it makes the credit assignment problem too hard for the agent during the early stages. The baseline gamma of `0.99` is clearly superior for this environment, as it provides a much better balance between immediate feedback and long-term goals, leading to vastly more efficient learning.

4 Effect of Network Architecture (Capacity)

We tested increasing the capacity of the Multi-Layer Perceptron (MLP) policy network, keeping all other parameters at the baseline.

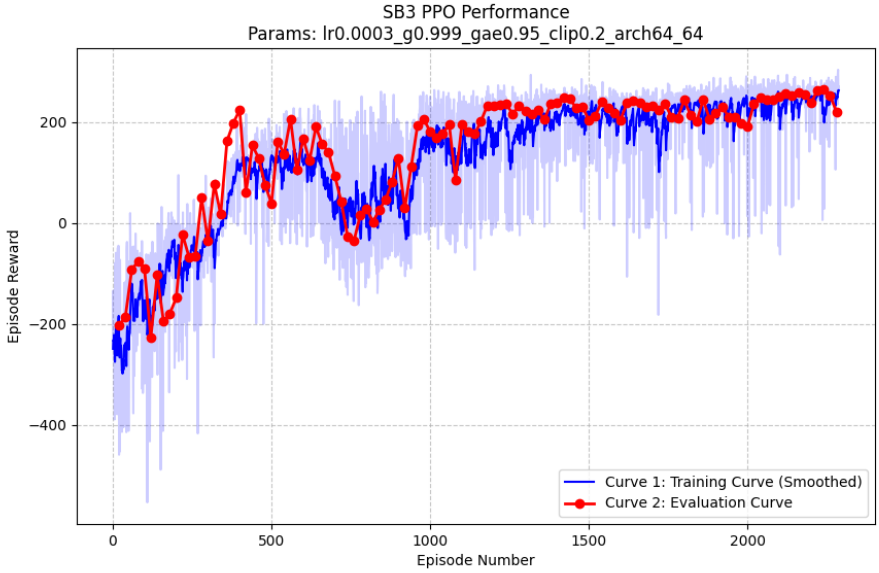

Figure 3: Performance with a higher discount factor (Gamma = 0.999).

- **Wider Network (`arch128_128`):** As seen in Figure 4, increasing the width significantly boosted both learning speed and stability. The agent reached the "solved" threshold (> 200) around episode 400 and maintained a very tight, stable performance curve thereafter.
- **Deeper Network (`arch64_64_64`):** Adding a third hidden layer (Figure 5) also accelerated learning compared to the baseline, but the curve showed slightly more noise and variance than the wider 128x128 network.

Conclusion: The default `[64, 64]` architecture may slightly underfit the continuous dynamics of the LunarLander. Increasing network capacity—especially network width (`[128, 128]`)—provides the best trade-off, allowing the agent to capture the complex state-action mappings more effectively without introducing excessive noise.

5 Effect of Clipping Boundary

Experiment: Tightened the PPO Clip Range from `0.2` to `0.1`.

Observation (`clip0.1`): Tightening the clipping boundary heavily restricts how much the policy can change in a single update step. As seen in Figure 6, the learning process is noticeably slower; it takes roughly 600-700 episodes to consistently cross the 100-reward mark. However, the conservative updates prevent the agent from making disastrous policy changes, resulting in a relatively stable curve once convergence is reached.

Conclusion: While a tighter clip range (0.1) prevents large drops in performance, it overly constrains exploration and slows down training. The default 0.2 appears to be a better balance for learning efficiency in this environment.

6 Summary

Through this hyperparameter tuning process, we observed the following key dynamics:

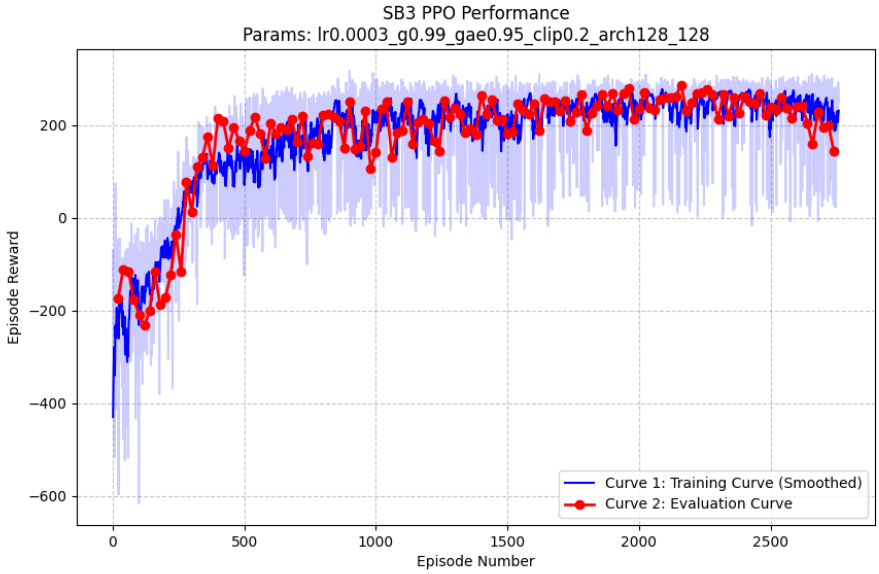

Figure 4: Performance with a wider network architecture [128, 128].

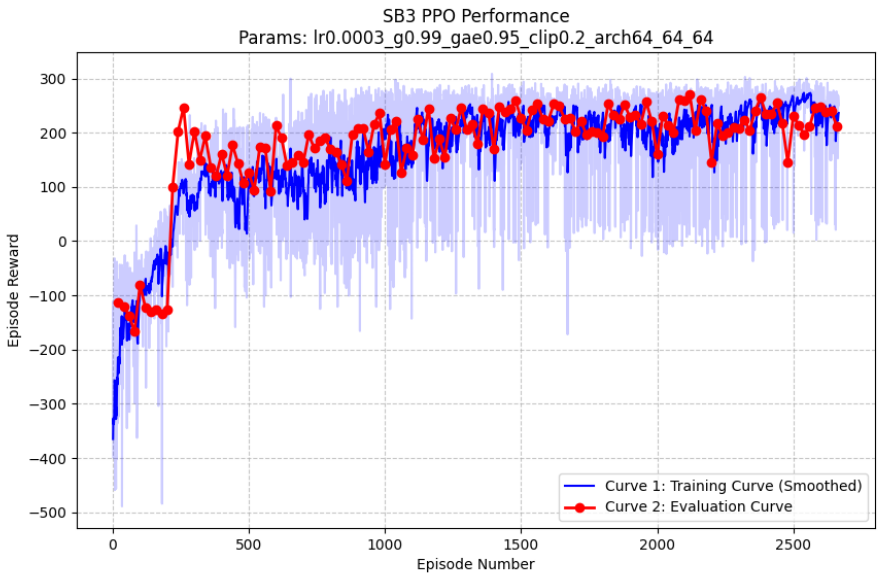

Figure 5: Performance with a deeper network architecture [64, 64, 64].

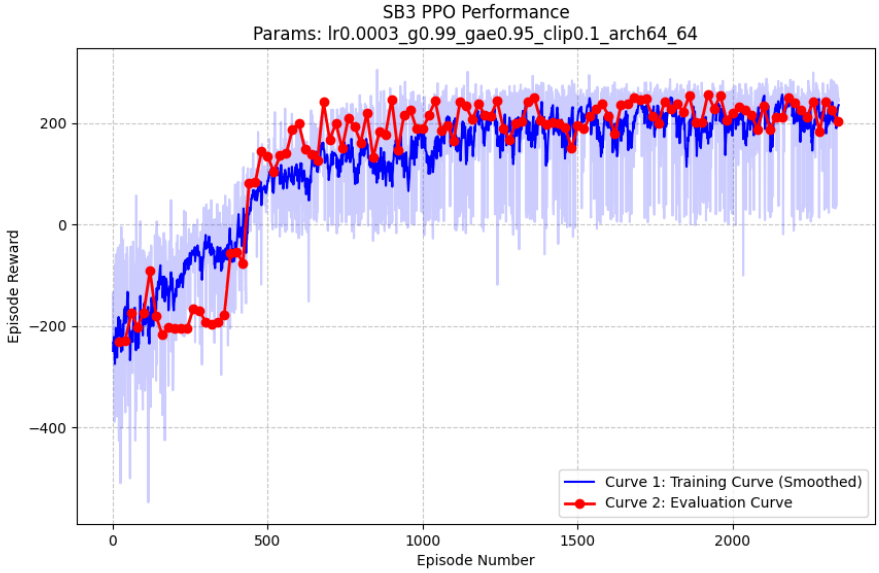

Figure 6: Performance with a tighter PPO clipping boundary of 0.1.

1. **Network Capacity** (`[128, 128]`) yields the most immediate and significant improvements in both learning speed and final performance **among the tested configurations**. However, blindly increasing the number of parameters is not universally beneficial. Overly large or deep networks can lead to overfitting, require significantly more environment interactions to train (sample inefficiency), and create a harder optimization landscape that may actually slow down convergence or lead to instability.

2. **Gamma** (`0.99`) provides the best balance for learning efficiency. An excessively high gamma (`0.999`) makes the initial learning phase far too difficult, despite any potential long-term stability.

3. **Aggressive Learning Rates** (`0.001`) or **Overly Restrictive Clipping** (`0.1`) are detrimental to the process, leading to high variance and sluggish learning, respectively.

Disclaimer: It is important to note that these conclusions are strictly bounded by the specific hyperparameter combinations and ranges explored in this assignment. Reinforcement learning environments often exhibit complex, non-linear interactions between parameters. An exhaustive or joint search (e.g., using Bayesian optimization) might reveal a different set of globally optimal hyperparameters.

If we were to construct a final "Optimized" agent based solely on these isolated findings, combining the `[128, 128]` network architecture with the baseline gamma (`0.99`) and the baseline learning rate (`0.0003`) would likely yield the most robust and efficient LunarLander model.